Pointers

Addresses, dereferencing, pointer arithmetic and pass by reference

Contents

Concepts	What is a pointer?, Basic operators, Printing addresses, Pass by value vs pass by reference
Example 1	Printing the address of variables
Example 2	Declaring and using a pointer
Example 3	Accessing array elements with a pointer
Example 4	Pointer arithmetic
Example 5	Pointer to a pointer
Example 6	Generic (void) pointer
Example 7	Swapping two values: by value vs by reference
Example 8	Increase a number by 10 through a pointer
Example 9	Sum of two floats using their addresses
Example 10	String length with a pointer (without strlen)

What is a pointer?

Every variable is stored at an **address** in memory. A pointer is a variable that stores such an address. Through a pointer you can read or change the variable it points to, even from inside another function.

Basic operators

Syntax	Meaning
`int *p;`	declares p as a pointer to an int
`&x`	address of x
`p = &x;`	p now points to x
`*p`	the value at the address in p (dereferencing)
`*(p + i)`	the element i positions after p (same as p[i])
`int **pp;`	pointer to a pointer
`void *vp;`	generic pointer: can hold any address, but must be cast before use

Printing addresses

- Use `%p` to print an address. Printing an address with `%d` or `%x` is not correct and may cut off part of it on 64-bit systems.
- The name of an array already means the address of its first element, so write `printf("%p", numbers);` without `&`.
- `&numbers[2]` and `numbers + 2` both give the address of the third element.
- Addresses change every time the program runs, so your output will not match the examples exactly.

Pass by value vs pass by reference

When you pass a normal variable to a function, the function receives a **copy**. Changes made to the copy disappear when the function returns. If you pass the variable's **address** instead, the function can change the original variable through the pointer. This is why pointers are needed for functions such as swap.

EXAMPLE 1

Printing the address of variables

Print the memory addresses of two variables and of each element of an int array and a double array.

CODE

```c
#include <stdio.h>

int main() {
    int x = 10, y = 20;
    int numbers[4] = {10, 20, 30, 40};
    double values[3] = {1.1, 2.2, 3.3};

    printf("Address of x: %p\n", (void *)&x);
    printf("Address of y: %p\n", (void *)&y);

    for (int i = 0; i < 4; i++) {
        printf("Address of numbers[%d]: %p\n", i, (void *)&numbers[i]);
    }
    for (int i = 0; i < 3; i++) {
        printf("Address of values[%d]: %p\n", i, (void *)&values[i]);
    }
    return 0;
}
```

SAMPLE RUN (addresses differ on every run)

```
Address of x: 0x7ffda279c7a0
Address of y: 0x7ffda279c7a4
Address of numbers[0]: 0x7ffda279c7b0
Address of numbers[1]: 0x7ffda279c7b4
Address of numbers[2]: 0x7ffda279c7b8
Address of numbers[3]: 0x7ffda279c7bc
Address of values[0]: 0x7ffda279c7c0
Address of values[1]: 0x7ffda279c7c8
Address of values[2]: 0x7ffda279c7d0
```

Note. Consecutive int addresses differ by 4 bytes and double addresses by 8 bytes, which are the sizes of these types. Casting to `(void *)` is the strictly correct way to pass an address to `%p`.

EXAMPLE 2

Declaring and using a pointer

Create a pointer to an int, then print the value and the address through the pointer.

CODE

```c
#include <stdio.h>

int main() {
    int x = 10;
    int *p = &x;            /* p stores the address of x */

    printf("x = %d\n", *p);
    printf("Address of x (via pointer): %p\n", (void *)p);
    printf("Address of x (via &x):      %p\n", (void *)&x);
    printf("x + 1 = %d\n", *p + 1);
    return 0;
}
```

SAMPLE RUN (addresses differ on every run)

```
x = 10
Address of x (via pointer): 0x7ffe84424b6c
Address of x (via &x):      0x7ffe84424b6c
x + 1 = 11
```

Note. `int *a = x;` would be a mistake: it stores the *value* 10 as if it were an address.

EXAMPLE 3

Accessing array elements with a pointer

Point a pointer at an array and print the first two elements.

CODE

```c
#include <stdio.h>

int main() {
    int arr[] = {1, 2, 3, 4, 5};
    int *ptr = arr;         /* points to arr[0] */

    printf("First element: %d\n", *ptr);
    printf("Second element: %d\n", *(ptr + 1));
    return 0;
}
```

SAMPLE RUN

```
First element: 1
Second element: 2
```

EXAMPLE 4

Pointer arithmetic

Print every element of a 10-element array by moving through it with a pointer.

CODE

```c
#include <stdio.h>

int main() {
    int numbers[10] = {10, 20, 30, 40, 50, 60, 70, 80, 90, 100};
    int *ptr = numbers;

    printf("Array elements:\n");
    for (int i = 0; i < 10; i++) {
        printf("%d ", *(ptr + i));      /* same as ptr[i] */
    }
    printf("\n");
    return 0;
}
```

SAMPLE RUN

```
Array elements:
10 20 30 40 50 60 70 80 90 100
```

> **Note.** `ptr + 1` does not add 1 byte. It moves forward by one *element*, which is 4 bytes for an int.

EXAMPLE 5

Pointer to a pointer

Store the address of a pointer in a second pointer and reach the original value with double dereferencing.

CODE

```c
#include <stdio.h>

int main() {
    int num = 5;
    int *ptr = &num;        /* ptr  -> num */
    int **ptr2 = &ptr;      /* ptr2 -> ptr -> num */

    printf("Value of num: %d\n", num);
    printf("Value via ptr: %d\n", *ptr);
    printf("Value via ptr2: %d\n", **ptr2);
    return 0;
}
```

SAMPLE RUN

```
Value of num: 5
Value via ptr: 5
Value via ptr2: 5
```

EXAMPLE 6

Generic (void) pointer

Write one function that can print either an int or a float. It receives a `void *` together with a type letter.

CODE

```c
#include <stdio.h>

void printValue(void *ptr, char type) {
    if (type == 'i') {
        printf("Integer value: %d\n", *(int *)ptr);       /* cast, then dereference */
    } else if (type == 'f') {
        printf("Float value: %.2f\n", *(float *)ptr);
    }
}

int main() {
    int num = 10;
    float fnum = 20.5;

    printValue(&num, 'i');
    printValue(&fnum, 'f');
    return 0;
}
```

SAMPLE RUN

```
Integer value: 10
Float value: 20.50
```

EXAMPLE 7

Swapping two values: by value vs by reference

Try to swap two variables with a function. The pass-by-value version only swaps local copies. The pass-by-reference version swaps the originals.

CODE

```c
#include <stdio.h>

void swapByValue(int a, int b) {        /* receives copies */
    int temp = a;
    a = b;
    b = temp;
}

void swapByReference(int *a, int *b) {  /* receives addresses */
    int temp = *a;
    *a = *b;
    *b = temp;
}

int main() {
    int x = 10, y = 20;

    swapByValue(x, y);
    printf("After swapByValue:     x = %d, y = %d\n", x, y);

    swapByReference(&x, &y);
    printf("After swapByReference: x = %d, y = %d\n", x, y);
    return 0;
}
```

SAMPLE RUN

```
After swapByValue:     x = 10, y = 20
After swapByReference: x = 20, y = 10
```

EXAMPLE 8

Increase a number by 10 through a pointer

Define an int with the value 5 in main, pass it to a function, and increase it by 10 inside the function.

CODE

```c
#include <stdio.h>

void addTen(int *p) {
    *p += 10;               /* changes the original variable */
}

int main() {
    int a = 5;

    printf("Before: %d\n", a);
    addTen(&a);
    printf("After: %d\n", a);
    return 0;
}
```

SAMPLE RUN

```
Before: 5
After: 15
```

EXAMPLE 9

Sum of two floats using their addresses

Pass the addresses of two float variables to a function that returns their sum.

CODE

```c
#include <stdio.h>

float sum(float *a, float *b) {
    return *a + *b;
}

int main() {
    float x = 4.44, y = 5.55;
    float result = sum(&x, &y);

    printf("Sum of the two numbers: %.3f\n", result);
    return 0;
}
```

SAMPLE RUN

```
Sum of the two numbers: 9.990
```

EXAMPLE 10

String length with a pointer (without strlen)

Count the characters of a string by moving a pointer forward until it reaches the null character `'\0'`.

CODE

```c
#include <stdio.h>

int stringLength(char *str) {
    int count = 0;

    while (*str != '\0') {      /* stop at the end-of-string marker */
        count++;
        str++;                  /* move to the next character */
    }
    return count;
}

int main() {
    char text[] = "Hello everyone";

    printf("String length: %d\n", stringLength(text));
    return 0;
}
```

SAMPLE RUN

```
String length: 14
```